

StockWise:

AI-Powered Decision System for Inventory Reordering and Waste Reduction

Deployment Plan

Field	Deployment Value
Frontend	Vercel Hobby plan, Next.js app from frontend/
Backend	Render Free web service, FastAPI from repository root
Persistence	Supabase for durable data; Render filesystem is ephemeral

1. Deployment Goal

StockWise should be deployed as a realistic initial production-ready deployment, not as a fully hardened production system. The goal is to make the application accessible for stakeholder evaluation, product demonstration, and controlled pilot testing while being honest about free-tier limits.

Current hosting model:

- Frontend: Vercel Hobby plan, using the `frontend` directory.
- Backend API: Render Free web service, using the repository root.
- Database/auth/persistence: Supabase project configured through environment variables.
- AI mode: `GLM_MODE=live` by default; `GLM_MODE=mock` only when invalid `ZAI_API_KEY` and provider configuration are unavailable.

This deployment is acceptable for judging and early validation. It is not suitable for paid customer production use without upgrading hosting, monitoring, and reliability controls.

2. Architecture Overview

```

User Browser
  |
  | HTTPS
  v
Vercel Frontend (Next.js, Hobby)
  |
  | NEXT_PUBLIC_API_BASE_URL
  v
Render Backend (FastAPI, Free Web Service)
  |
  | SUPABASE_URL / SUPABASE_SERVICE_ROLE_KEY
  v
Supabase (auth, records, analysis snapshots)
  |
  | optional, only in live AI mode
  v
Z.AI / ILMU GLM endpoint

```

The frontend should never call Supabase with the service-role key. Browser code may only use public Supabase values such as `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY`. Backend-only secrets stay on Render.

3. Free-Tier Constraints and Expected Behaviour

Vercel Hobby

Vercel Hobby is suitable for a student frontend, preview deployments, automatic HTTPS, and Git-based CI/CD. It is still a free/personal plan, so it should be treated as demo infrastructure. If the app becomes a commercial pilot, team project, or receives sustained traffic, move to a paid Vercel plan.

Important implications:

- Preview deployments are useful for judging or testing branches.
- Environment variable changes require a new deployment before they affect the live app.
- Frontend logs and observability are limited compared with paid plans.
- Avoid using the frontend as a heavy serverless backend; keep backend work on Render/FastAPI.

Render Free Backend

Render Free is the biggest reliability limitation. Free web services spin down after idle time and need time to wake up on the next request. For StockWise, this means the first API call after inactivity may be slow or appear stuck for around a minute.

Important implications:

- Judges should be told that the first backend request may be slow because the backend is waking up.
- The frontend should keep friendly loading and retry states.
- Do not store uploaded files, generated reports, SQLite databases, or important state on the Render filesystem. The filesystem is ephemeral.
- Use Supabase for durable records, import batches, and analysis snapshots.
- Do not describe this as paid-tier production infrastructure.

4. Environment Strategy

Local Development

Use local development for feature work and debugging.

Frontend:

```
NEXT_PUBLIC_API_BASE_URL=http://localhost:8000
NEXT_PUBLIC_SUPABASE_URL=http://127.0.0.1:54321
NEXT_PUBLIC_SUPABASE_ANON_KEY=public-anon-key-placeholder
```

Backend:

```
STOCKWISE_SUPABASE_ENABLED=false
GLM_MODE=live
```

Evaluation / Pilot Deployment Environment

Use this for stakeholder evaluation and product demonstration

Frontend on Vercel:

```
NEXT_PUBLIC_API_BASE_URL=https://your-render-service.onrender.com
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-supabase-anon-key
```

```

NEXT_PUBLIC_FRONTEND_URL=https://your-vercel-project.vercel.app
NEXT_PUBLIC_API_DISPLAY_URL=https://your-render-service.onrender.com
NEXT_PUBLIC_GITHUB_REPO=https://github.com/your-org-or-user/your-repo

```

Backend on Render:

```

STOCKWISE_CORS_ORIGINS=https://your-vercel-project.vercel.app
STOCKWISE_SUPABASE_ENABLED=true
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_SERVICE_ROLE_KEY=your-supabase-service-role-key
GLM_MODE=live
ZAI_API_KEY=

```

Use `GLM_MODE=live` for the main judging deployment if the live AI provider has been tested shortly before the demo. Mock mode is more stable and still demonstrates the product flow safely.

Live AI Demo Mode

Enable this only after the normal deployment is already healthy.

```

GLM_MODE=live
ZAI_API_KEY=your-provider-key
ZAI_BASE_URL=https://api.ilmu.ai/v1/chat/completions
ZAI_MODEL=ilmu-glm-5.1
ZAI_STREAM=false

```

Live AI mode has higher demo risk because provider latency, quota, malformed model output, and networking issues can affect the experience. The app includes parser validation and deterministic fallback, but judging demos should not depend completely on live AI.

5. Backend Deployment on Render

The repository already includes `render.yaml`:

```

services:
- type: web
  name: stockwise-backend
  runtime: python
  plan: free

```

```
buildCommand: pip install -r requirements.txt
startCommand: PYTHONPATH=src python -m uvicorn --factory
stockwise_api.api.app:create_app --host 0.0.0.0 --port $PORT
```

Recommended setup:

1. Create a Render web service from the repository or use the Blueprint.
2. Keep `Root Directory` as the repository root.
3. Confirm the service uses Python 3.11.9, matching `render.yaml`.
4. Set backend environment variables in Render dashboard.
5. Set the health check path to `/health` if configuring through the dashboard. If managing fully through Blueprint, add `healthCheckPath: /health` after confirming the service deploys correctly.
6. Deploy the latest commit.
7. Open <https://your-render-service.onrender.com/health>.

Expected `/health` values for demo readiness:

- `status = ok`
- `glm_mode = mock` for stable demo, or `live` only if live AI was tested
- `supabase_enabled = true`
- `supabase_store_ready = true`
- `snapshot_write_mode = required`
- `history_snapshot_table = analysis_source_observations`

If the first request is slow, wait for the Render Free service to wake up, then refresh.

6. Frontend Deployment on Vercel

Recommended setup:

1. Import the Git repository into Vercel.
2. Set `Root Directory` to `frontend`.
3. Use the Next.js framework preset.
4. Use install command:

```
npm ci
```

5. Use build command:

```
npm run build
```

6. Set the frontend environment variables in Vercel.
7. Deploy.
8. Copy the Vercel production URL into Render `STOCKWISE_CORS_ORIGINS`.
9. Redeploy the backend after updating CORS.

If the frontend shows CORS or API connection errors, check these in order:

10. `NEXT_PUBLIC_API_BASE_URL` points to the Render URL and does not include a trailing path.
11. Render `STOCKWISE_CORS_ORIGINS` exactly matches the Vercel URL.
12. The Render backend has been redeployed after the CORS update.
13. `/health` is reachable from the browser.

7. Deployment Sequence

Use this sequence for a clean demo deployment.

1. Run local checks:

```
pytest
cd frontend
npm run build
npm test -- --watch=false --runInBand
```

2. Push the latest commit to the deployment branch.
3. Deploy backend on Render first.
4. Check backend `/health`.
5. Deploy frontend on Vercel.
6. Update Render CORS with the final Vercel URL.
7. Redeploy backend.
8. Open the frontend and run the smoke test checklist.
9. Capture screenshots or notes for judging evidence.

8. Smoke Test Checklist

Run these after every deployment or environment-variable change.

Backend:

- Open `/health` and confirm expected fields.
- Upload a small valid CSV through the API or frontend.
- Confirm the response includes `analysis_id`, `dataset_summary`, `kpi_summary`, and `items`.
- Confirm logs do not show `stockwise.analysis_snapshot.failure`.

Frontend:

- Open the Vercel URL.
- Upload a valid CSV or use manual entry.
- Confirm dashboard KPI cards and ranked item table render.
- Open Records Review and confirm source observations are visible.
- Run one item simulation.
- Open explanation or AI decision brief.
- Open Export Analysis and confirm Business Value Snapshot renders.

Persistence:

- Refresh the dashboard route after upload.
- If possible, restart or redeploy Render, then reload the same analysis.
- Confirm the dashboard can recover from Supabase snapshot data.

Free-tier demo readiness:

- Wake the Render backend 2-3 minutes before judging.
- Keep `/health` open in another tab during demo setup.
- If the backend sleeps during a long waiting period, explain the cold start honestly.

9. Rollback and Recovery

For an initial production-ready deployment, keep rollback simple.

Frontend rollback:

- Use Vercel Deployments to promote the previous working deployment.
- If the issue is an environment variable, fix it and redeploy.

Backend rollback:

- Use Render Manual Deploy to deploy a previous known-good commit.

- If the service fails after an environment change, restore the previous env values and redeploy.
- If live AI mode causes errors, immediately switch `GLM_MODE=mock` and redeploy.

Data recovery:

- Do not rely on Render local files.
- Supabase snapshot tables are the durable source for analysis recovery.
- For serious data issues, stop demos that depend on persistence and use mock/offline demo data until the problem is understood.

10. Monitoring and Evidence

Minimum monitoring for the current free-tier deployment:

- Render logs for backend startup errors, CORS errors, Supabase errors, and `stockwise.analysis_snapshot.* events`.
- Vercel deployment logs for frontend build failures.
- Browser console/network tab for frontend API and CORS issues.
- `/judge-mode` page, if available, for showing health metadata during judging.
- Manual smoke test evidence after each deployment.

Recommended demo evidence:

- Screenshot of `/health`.
- Screenshot of successful dashboard after upload.
- Screenshot of Records Review showing source observations.
- Screenshot of Simulation plus AI Trade-off Verdict.
- Screenshot of Export Analysis Business Value Snapshot.

11. Security and Secrets

Rules:

- Never commit `.env`, `.env.local`, service-role keys, API keys, or database dumps.
- Keep `SUPABASE_SERVICE_ROLE_KEY` only on Render.
- Keep `ZAI_API_KEY` only on Render.
- Only expose `NEXT_PUBLIC_*` values to Vercel frontend.
- Rotate keys if they appear in screenshots, logs, commits, or shared documents.

- Use mock AI mode for public demos if live API cost or key exposure is a concern.

12. Upgrade Triggers

Stay on free tiers for judging and early student validation. Upgrade when any of these become true:

- Users complain about cold starts or demo reliability.
- The app is used by real businesses beyond friendly pilots.
- You need stable uptime during business hours.
- You need stronger logs, monitoring, team access, or custom domains.
- AI/provider calls become frequent enough that usage limits and latency matter.
- You start charging customers.

Suggested first upgrades:

1. Upgrade Render backend to a paid instance to remove cold starts.
2. Keep Vercel Hobby until usage or commercial/team requirements force an upgrade.
3. Add basic uptime monitoring and a scheduled health check only after checking platform rules and cost.
4. Add a staging environment when more than one person is deploying regularly.

13. Honest Deployment Statement for Judges

StockWise is currently deployed as an initial production-ready deployment using Vercel Hobby for the frontend and Render Free for the FastAPI backend. This keeps cost low and is suitable for hackathon evaluation and small pilot testing. The main known limitation is Render Free cold start behaviour: the first backend request after inactivity can be slow because the service wakes from sleep. Durable application data is stored in Supabase, not on Render's filesystem. The current recommended judging configuration uses deterministic scoring for reliability, with live GLM mode available only after a separate smoke test.